



SafeNode Operations and Incident Response

Production procedures for deployment, monitoring, account incidents, data incidents, and recovery.

Document version: 2026-07-27

Product: Safenode

Classification: Internal / customer-safe where noted

Production topology

Service	Platform	Primary check
Frontend	Cloudflare Pages / safe-node.app	HTTP 200, asset loading, security headers
API	Railway / api.safe-node.app	/health and /api/health/ready
Database	Railway Postgres via Prisma	Schema sync and connection health
Email	Resend / mail.safe-node.app	Domain verified, delivery event visible
Billing	Paddle	Checkout, signed webhook, subscription state
Telemetry	Sentry	Backend/frontend events and alert routing

Deployment checklist

- Confirm the merge commit is on main and CI is green.
- Confirm Railway environment variables are present without printing their values.

- Run Prisma db push through the Railway pre-deploy command.
- Confirm Railway health check and logs show the expected production process.
- Build frontend with VITE_API_URL and VITE_MOBILE_API_URL set to the production API.
- Deploy Pages and verify the canonical domain, association files, and headers.
- Run smoke tests for auth, vault, email verification, billing, and passkey flows.
- Tag the release only after production verification.

Incident severity

Severity	Example	Initial action
P0	Secret leakage, vault crypto regression, mass auth bypass	Stop release, rotate secrets, preserve evidence, notify owners
P1	Production auth/billing/email outage, device lockout regression	Mitigate within hours, rollback or disable affected feature
P2	Single workflow failure or degraded telemetry	Create issue, workaround, fix in normal release
P3	Copy/UI/documentation defect	Schedule with product maintenance

Credential or secret incident

- Disable affected integration or rotate the credential immediately.
- Identify exposure window from Git history, CI logs, provider logs, and Sentry.
- Rotate JWT signing secret and encryption key according to the data migration plan; do not blindly replace an encryption key if existing data still requires it.
- Invalidate sessions if token signing material was exposed.
- Purge public artifacts and document the commit/provider action taken.
- Run a post-incident review and add a regression control.

Data or vault incident

- Do not access or export customer plaintext. Preserve encrypted records and metadata only.
- Determine whether the event affects ciphertext, keys, authentication, devices, or availability.
- If a client-side key may be exposed, force session lock/re-authentication and guide the user through passkey/recovery rotation.
- Communicate honestly: what was exposed, what was not, impact, and required user action.

Rollback and recovery

- Frontend: redeploy the last known-good Pages artifact.
- Backend: redeploy the last known-good Railway commit; preserve database schema compatibility.
- Database: use provider backups and tested restore procedures; do not run destructive SQL during an active incident without approval.
- After recovery, validate health, auth, vault read/write, webhook idempotency, and email delivery.